

CUDA Jacobi 集项目验收演示文档

本文档供验收现场展示使用，仅包含**演示命令**及对**输入/输出**的说明，不含口头解说词。

演示目录（所有命令均在此目录下执行）：

```
cd "/mnt/d/金介然/大三下/gpu/大作业/CODE/mycode"
```

标准测试网格： ../JacobiSetComputation-master/test_torus.obj

1. 项目背景与实现说明

1.1 项目概述

项目	说明
任务	对照串行工程 JacobiSetComputation-master ，实现 Jacobi 集逐边判定的 CUDA 并行版本
论文基础	<i>Jacobi Sets of Multiple Morse Functions</i> （Edelsbrunner 等）
串行参考	CODE/JacobiSetComputation-master/build/JacobiSetComputation
并行实现	CODE/mycode/build/jacobi_cuda
完整分析报告	docs/analysis/cuda_jacobi_code_analysis.md

1.2 问题定义

在二维三角网格（嵌入 3D 空间）上，给定两个定义在顶点上的标量函数 f 、 g ，求 **Jacobi 集** 对应的网格边集合。

本项目的默认测试函数与串行版一致：

- $f = y$ （顶点 y 坐标）

- $g = x$ （顶点 x 坐标）

标准测试网格 `test_torus.obj` 上，Jacobi 集为 **64 条边**，在 ParaView 中呈现为 torus 内外两圈闭合曲线。

1.3 串行算法核心（对照基准）

串行工程 `JacobiSet::compute()` 对每条**内部边** ($e1, e2$) 执行：

1. 通过 `get_e_link()` 找到 link 中两个对顶点 $v1$ 、 $v2$
2. 分别计算四个 lower-link 布尔值：
 - `f_lower_v1 = is_lowerLink(e1, e2, v1, g)`
 - `f_lower_v2 = is_lowerLink(e1, e2, v2, g)`
 - `g_lower_v1 = is_lowerLink(e1, e2, v1, f)`
 - `g_lower_v2 = is_lowerLink(e1, e2, v2, f)`
3. 判定临界条件：`f_crit = (f_lower_v1 == f_lower_v2)`
4. 若 `f_crit` 为真，该边属于 Jacobi 集

`is_lowerLink` 内部组合 **POS1** ((f,g) 平面方向) 与 **POS2** (单函数大小比较)，二者一致则返回 `true`。

并行化依据：每条边的判定只依赖 ($e1, e2, v1, v2$) 及函数值，边与边之间无数据依赖。

1.4 并行化思路

为什么选择「边」为并行粒度

步骤	是否适合 GPU	原因
读 mesh、建邻接	否	I/O 与复杂 C++ 容器（ <code>trimesh2</code> ）
逐边 Jacobi 判定	是	数量大、流程相同、无依赖
SoS 符号计算	否	CPU 库，移植成本高
写结果	否	I/O

CUDA 映射

内部边数组 index $i \rightarrow$ CUDA thread i

每个线程读取一条 EdgeRecord ，执行与串行相同的 lower-link 判定，写入 results[i] 。

CPU-GPU 混合方案

CPU： mesh 读取 + 边表预处理 + SoS 退化回退 + 结果写出

GPU： 内部边并行 Jacobi 判定 (compute_jacobi_kernel)

未将整个程序搬到 GPU，而是只并行化算法中 $O(E)$ 的核心判定循环 (E 为内部边数)。

1.5 实现架构

工程文件

文件	职责
main_cuda.cpp	CPU 主程序：读 mesh、预处理边表、调 GPU、SoS 回退、写结果
jacobi_cuda.cu	CUDA device 函数 + kernel + host 封装 (compute_jacobi_gpu)
jacobi_gpu.h	CPU/GPU 共享数据结构
CMakeLists.txt	构建配置，链接 trimesh2、SoS 及串行工程 TriMeshJ.cpp、JacobiSet.cpp

数据结构

GPU 输入 (jacobi_gpu.h)：

```
struct EdgeRecord {
    int e1, e2;          // 边端点
    int link1, link2;    // link 对顶点
};
```

GPU 输出：

```
struct JacobiGpuResult {
    int fv1, fv2;      // 边端点
    int min_f, min_g, pos_align; // 与串行 JacobiEdge 对应
    int is_jacobi;      // 是否属于 Jacobi 集
    int degenerate;     // 是否触发近退化，需 CPU SoS 回退
};
```

串行版对 Jacobi 边 push_back；并行版为每条内部边预分配固定槽位 results[idx]，避免 GPU 多线程写冲突。

完整数据流

- 读 test_torus.obj
- CPU: need_edges() + need_neighbors() → EdgeRecord[] 内部边表
- 构造 f=y, g=x
- cudaMemcpy → GPU
- compute_jacobi_kernel: 每线程 4 次 device_is_lower_link → f_crit → is_jacobi
- cudaMemcpy ← CPU
- apply_sos_fallback(): 对 degenerate==1 的边调用串行 SoS 覆盖
- write_jacobi_edges() → *_gpu_jacobi.txt
- compare_jacobi.py 与串行 test_torus_jacobi.txt 对拍

退化处理（GPU + CPU SoS 混合）

路径	处理方式
普通边	GPU double 判定（device_pos1 / device_pos2）
近退化边	GPU 标记 degenerate=1，CPU 调用串行 JacobiSet::is_lowerLink()（SoS）覆盖

torus 测试：1536 条内部边中 16 条退化，其余由 GPU 并行完成。

与串行的主要差异

维度	串行	mycode
执行	CPU 逐边循环	GPU 一线程一边
link 获取	循环内 get_e_link()	CPU 预处理为 EdgeRecord[]
退化	全程 SoS	GPU 快判 + CPU SoS 回退

维度	串行	mycode
输出	满足条件才 push_back	全量 results[] ，写文件时过滤

判定公式本身（POS1、POS2、is_lowerLink、f_crit）与串行 **完全一致**。详见 docs/analysis/cuda_jacobi_core_differences.md 。

1.6 第三方依赖

依赖	作用
trimesh2	读 .obj 、网格拓扑；扩展为 TriMeshJ （边、link）
Detri / SoS	退化情况下精确符号判定；用于 CPU SoS 回退
OpenMP	链接 trimesh2 静态库所需

安装脚本： setup_deps_wsl.sh （首次环境配置时使用）。

2. 环境检查

2.1 CUDA 编译器

命令：

```
nvcc --version
```

预期输出（关键行）：

```
Cuda compilation tools, release 12.0, V12.0.140
```

说明： 确认 WSL 中 CUDA Toolkit 可用，后续 build_cuda_wsl.sh 依赖 nvcc 。

2.2 GPU 运行环境

命令：

```
nvidia-smi
```

预期输出（关键行）：

```
GPU Name: NVIDIA GeForce RTX 4060  
CUDA Version: 13.0
```

说明：

- `nvidia-smi` 中的 **CUDA Version** 为驱动支持的最高运行时版本
- `nvcc --version` 为实际安装的 Toolkit 版本
- 驱动版本高于 Toolkit 版本属正常情况，可向下兼容

3. 构建

命令：

```
bash build_cuda_wsl.sh
```

输入：CMakeLists.txt、main_cuda.cpp、jacobi_cuda.cu 及串行工程依赖库

预期输出（关键行）：

```
-- Configuring done  
-- Generating done  
[100%] Built target jacobi_cuda  
Built: /mnt/d/金介然/大三下/gpu/大作业/CODE/mycode/build/jacobi_cuda
```

说明：生成可执行文件 `build/jacobi_cuda`，链接 `trimesh2`、`SoS` 及串行工程中的 `TriMeshJ.cpp`、`JacobiSet.cpp`。

4. 功能正确性测试

4.1 CPU 预处理（边表生成）

命令：

```
bash test_preprocess.sh
```

等价手动命令：

```
./build/jacobi_cuda --preprocess-only ../JacobiSetComputation-master/test_torus.obj
```

输入： test_torus.obj （torus 三角网格）

预期输出：

```
vertices: 512
edges: 1536
faces: 1024
interior_edges: 1536
```

输出字段说明：

字段	含义
vertices	网格顶点数
edges	无向边总数
faces	三角面数
interior_edges	内部边数（link 完整的边，供 GPU 使用）

说明： 本 torus 网格所有边均为内部边，故 interior_edges == edges 。此步骤验证 CPU 端 need_edges() 、 get_e_link() 及 EdgeRecord 生成正确，不调用 GPU。

4.2 GPU 基础计算

命令：

```
bash test_gpu_basic.sh
```

等价手动命令：

```
./build/jacobi_cuda ../JacobiSetComputation-master/test_torus.obj --output /tmp/test_torus_gpu_:
```

输入：

- 网格： test_torus.obj
- 函数： $f = y$, $g = x$ （顶点坐标，与串行版一致）

预期输出（关键行）：

```
jacobi_edges: 64
degenerate_edges: 16
sos_fallback_edges: 16
gpu_kernel_ms: <浮点数>
gpu_total_ms: <浮点数>
output: /tmp/test_torus_gpu_jacobi.txt
```

输出字段说明：

字段	含义
jacobi_edges	最终 Jacobi 边数量 (torus 标准答案：64)
degenerate_edges	GPU 判定中触发近退化的边数
sos_fallback_edges	经 CPU SoS 回退重新计算的边数
gpu_kernel_ms	CUDA kernel 纯执行时间 (ms)
gpu_total_ms	含显存分配、数据传输、kernel、结果拷回的总 GPU 调用时间 (ms)
output	Jacobi 边结果文件路径

说明： GPU 对所有内部边并行执行 lower-link 判定；退化边由 CPU SoS 覆盖，保证与串行 SoS 版一致。

4.3 退化边导出

命令：

```
bash test_degenerate_dump.sh
```

等价手动命令：

```
./build/jacobi_cuda ../JacobiSetComputation-master/test_torus.obj \  
--output /tmp/test_torus_gpu_jacobi.txt \  
--dump-degenerate /tmp/test_torus_degenerate_edges.txt
```

预期输出（关键行）：

```
degenerate_edges: 16  
sos_fallback_edges: 16  
degenerate_dump: /tmp/test_torus_degenerate_edges.txt
```

输出文件格式（ /tmp/test_torus_degenerate_edges.txt ）：

```
# edge_id e1 e2 link1 link2 is_jacobi  
0 0 1 17 496 0  
...
```

说明：共 17 行（1 行表头 + 16 条退化边）。 `edge_id` 为内部边表下标，对应 GPU 结果数组中的位置。

4.4 SoS 回退

命令：

```
bash test_sos_fallback.sh
```

预期输出（关键行）：

```
jacobi_edges: 64
degenerate_edges: 16
sos_fallback_edges: 16
output: /tmp/test_torus_gpu_sos_jacobi.txt
```

说明：

- sos_fallback_edges: 16 表示 16 条退化边均已调用串行 JacobiSet::is_lowerLink()（SoS 分支）重新计算
- 最终 Jacobi 边数仍为 64，输出文件供后续与串行参考比对

5. 串行与并行结果比对

5.1 边集比对（顺序无关）

命令：

```
bash test_compare_jacobi.sh
```

比对文件：

角色	路径
串行参考	../JacobiSetComputation-master/test_torus_jacobi.txt
CUDA 结果	/tmp/test_torus_gpu_sos_jacobi.txt

预期输出：

```
reference_edges: 64
candidate_edges: 64
match: yes
```

输出字段说明：

字段	含义
reference_edges	串行 SoS 版 Jacobi 边数

字段	含义
candidate_edges	CUDA hybrid 版 Jacobi 边数
match	边集合是否完全一致（yes 为通过）

说明： compare_jacobi.py 将每条边规范化为（min_vertex, max_vertex）后做集合比较，不依赖输出顺序。

5.2 文本文件逐字节比对

命令：

```
diff -u "../JacobiSetComputation-master/test_torus_jacobi.txt" /tmp/test_torus_gpu_sos_jacobi.txt
```

预期结果： 无输出（表示两文件内容完全一致）

说明： 除边集合一致外，当前实现输出顺序也与串行参考相同。

6. 性能测试

6.1 Benchmark 冒烟测试

命令：

```
bash test_benchmark_smoke.sh
```

预期输出示例：

```
32x16: cpu_wall=... ms, gpu_wall=... ms, speedup=...x, match=yes
wrote: /tmp/jacobi_benchmark_smoke.csv
```

说明： 小规模（32×16 torus）验证 benchmark 脚本可运行，且结果 match=yes 。

6.2 多规模性能测试

命令：

```
python3 benchmark_jacobi.py \
  --sizes 32x16,64x32,128x64,256x128,512x256 \
  --output /tmp/jacobi_benchmark.csv

cat /tmp/jacobi_benchmark.csv
```

输入：不同 (u,v) 参数的 torus 网格（由 mesh_make 自动生成）

CSV 字段说明：

字段	含义
u,v	torus 网格参数
vertices,edges	网格规模
jacobi_edges	Jacobi 边数
degenerate_edges	退化边数
sos_fallback_edges	SoS 回退边数
cpu_wall_ms	串行程序端到端耗时
gpu_wall_ms	CUDA 程序端到端耗时
gpu_kernel_ms	GPU kernel 耗时（程序内部统计）
gpu_total_ms	GPU 调用总耗时（程序内部统计）
speedup_wall	cpu_wall_ms / gpu_wall_ms
match	串行/GPU 边集是否一致

参考结果（验收环境可能略有浮动，以 match=yes 为准）：

u,v,vertices,edges,jacobi_edges,degenerate_edges,sos_fallback_edges,cpu_wall_ms,gpu_wall_ms,gpu_32,16,512,1536,64,16,16,~45,~274,~2,~5,~0.17,yes
64,32,2048,6144,128,32,32,~94,~247,~1,~4,~0.38,yes
128,64,8192,24576,256,64,64,~190,~288,~1,~4,~0.66,yes
256,128,32768,98304,512,128,128,~520,~465,~1,~5,~1.12,yes
512,256,131072,393216,1024,260,260,~1802,~1115,~1,~7,~1.62,yes

说明：

- 所有规模 match=yes ，正确性优先
- 小规模端到端加速比 < 1：固定开销（I/O、预处理、传输、SoS 初始化）占主导
- 大规模（ $\geq 256 \times 128$ ）端到端加速比 > 1
- gpu_kernel_ms 始终约 1–2 ms，核心逐边判定在 GPU 上执行较快

7. 可视化

7.1 生成 VTK 文件并复制到项目目录

步骤 1：生成 VTK

```
python3 jacobi_to_vtk.py /tmp/test_torus_gpu_sos_jacobi.txt
```

输入： Jacobi 边文本文件（格式： JacobiSet 头 + 边数 + 每行两端点坐标）

预期输出：

```
Wrote /tmp/test_torus_gpu_sos_jacobi.vtk
```

默认写在 /tmp/ ，WSL 外（Windows ParaView）不方便直接打开，需复制到项目文件夹。

步骤 2：复制到 CODE/mycode （便于 Windows 下 ParaView 打开）

```
cp /tmp/test_torus_gpu_sos_jacobi.vtk \  
"/mnt/d/金介然/大三下/gpu/大作业/CODE/mycode/test_torus_gpu_sos_jacobi.vtk"
```

复制后路径：

环境	路径
WSL	/mnt/d/金介然/大三下/gpu/大作业/CODE/mycode/test_torus_gpu_sos_jacobi.vtk
Windows	D:\金介然\大三下\gpu\大作业\COD\mycode\test_torus_gpu_sos_jacobi.vtk

说明：叠加显示 torus 网格时，mesh 文件已在串行工程目录，无需再复制：

D:\金介然\大三下\gpu\大作业\COD\JacobiSetComputation-master\test_torus.obj

7.2 ParaView 查看要点

打开方式（Windows ParaView）：

- 1. File → Open → 选择 COD\mycode\test_torus_gpu_sos_jacobi.vtk → Apply
- 2. 再 Open → 选择 COD\JacobiSetComputation-master\test_torus.obj → Apply

观察项	预期现象
单独加载 Jacobi VTK	内外两条闭合曲线
与 torus 网格叠加	曲线贴合网格表面
测试函数 f=y, g=x	Jacobi 集位于 $z \approx 0$ 截面附近，呈内外两圈

8. 完整演示命令（时间充足）

```
cd "/mnt/d/金介然/大三下/gpu/大作业/CODE/mycode"

# 环境
nvcc --version
nvidia-smi

# 构建
bash build_cuda_wsl.sh

# 功能测试
bash test_preprocess.sh
bash test_gpu_basic.sh
bash test_degenerate_dump.sh
bash test_sos_fallback.sh

# 正确性比对
bash test_compare_jacobi.sh
diff -u "../JacobiSetComputation-master/test_torus_jacobi.txt" /tmp/test_torus_gpu_sos_jacobi.txt

# 性能
bash test_benchmark_smoke.sh
python3 benchmark_jacobi.py --sizes 32x16,64x32,128x64,256x128,512x256 --output /tmp/jacobi_benchmark.csv
cat /tmp/jacobi_benchmark.csv

# 可视化
python3 jacobi_to_vtk.py /tmp/test_torus_gpu_sos_jacobi.txt
cp /tmp/test_torus_gpu_sos_jacobi.vtk \
  "/mnt/d/金介然/大三下/gpu/大作业/CODE/mycode/test_torus_gpu_sos_jacobi.vtk"
```

9. 最小演示命令（时间有限）

```
cd "/mnt/d/金介然/大三下/gpu/大作业/CODE/mycode"
bash build_cuda_wsl.sh
bash test_preprocess.sh
bash test_gpu_basic.sh
bash test_sos_fallback.sh
bash test_compare_jacobi.sh
```

覆盖范围：构建 → 预处理 → GPU 计算 → SoS 回退 → 与串行结果一致（match: yes）

10. 标准测试数据汇总（test_torus.obj）

指标	数值
顶点数	512
边数	1536
面数	1024
内部边数	1536
Jacobi 边数	64
退化边数	16
串行/GPU 边集比对	match: yes
diff 文本比对	无差异

11. 相关文件索引

文件	路径
CUDA 主程序	CODE/mycode/main_cuda.cpp
CUDA kernel	CODE/mycode/jacobi_cuda.cu

文件	路径
数据结构	CODE/mycode/jacobi_gpu.h
串行 Jacobi 判定	CODE/JacobiSetComputation-master/src/JacobiSet.cpp
串行参考输出	CODE/JacobiSetComputation-master/test_torus_jacobi.txt
完整分析报告	docs/analysis/cuda_jacobi_code_analysis.md
串行/并行差异对照	docs/analysis/cuda_jacobi_core_differences.md
GPU Jacobi VTK（可视化）	CODE/mycode/test_torus_gpu_sos_jacobi.vtk（§7.1 复制生成）
torus 网格（叠加用）	CODE/JacobiSetComputation-master/test_torus.obj

文档用途：验收现场演示；不含口头解说词。